



CPU EDUCATION

COMP90051

期末班

□ 4 讲解

TUTOR: JJ 学长

时长: 2 小时

本周内容预览:

1. Feed-Forward Neural Network

2. Autoencoder

3. CNN

4. RNN



* (Backpropagation)





CPU EDUCATION



关于CPU EDUCATION

CPU Education成立于2018年，总部在澳大利亚墨尔本，目前国内办公室在广州。经过近多年的积累与发展，公司凭借优质的教学质量和专业的研发团队力量不断壮大，成为全球最受学生欢迎的教育机构之一，在众多学生及老师中享有良好的口碑。CPU Education争做行业标杆，为不同阶段的学生提供专业，专属，专注的教学服务。

致力于为学生实现人人H1的目标，以“为学生提供极具价值的教学服务，培训各行业精英”为宗旨。秉承“教育高于一切”的经营理念，本着服务定制化、师资专业化、教学规范化的要求，打造墨尔本教育行业的“黄埔军校”。

课后，如果您有任何建议和意见，我们都非常欢迎您联系小助手分享您的想法，给予我们改进和提高的机会！感谢您参与CPU Education的辅导课程！

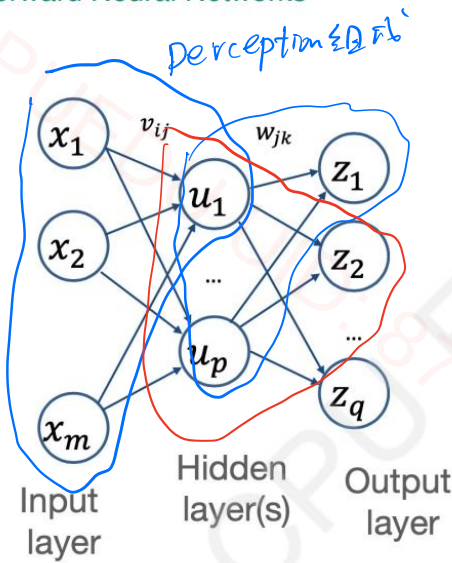


课程反馈



PART 1 Feed Forward Neural Network

Feedforward Neural Networks



Activation function

$$r_j = \sum_{i=0}^m x_i v_{ij}$$

Activation function

$$u_j = g(r_j)$$

Activation function

$$s_k = \sum_{j=0}^p u_j w_{jk}$$

Activation function

$$z_k = h(s_k)$$

Can deal with non-linear data given

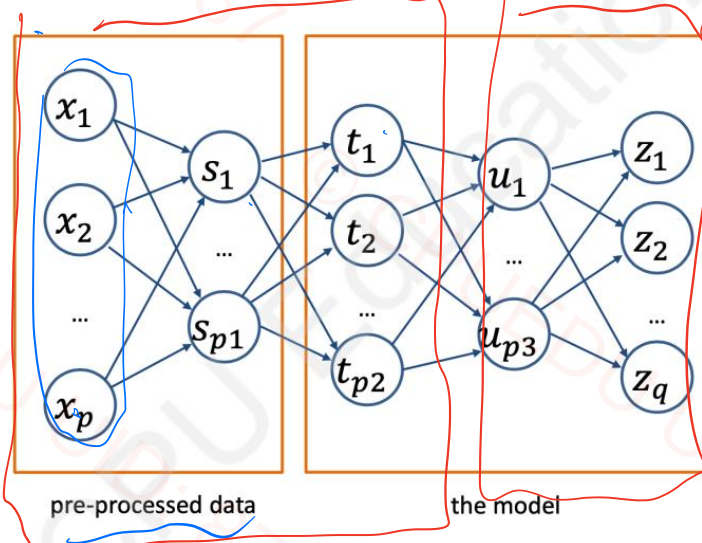
- at least one hidden layer
- non-linear activation function

Feedforward Neural Networks: see layers as "automatic feature extraction"

$[x_1, \dots, x_p]$

↓

$[s_1, \dots, s_{p1}]$



Data transformation is a "learnable function" (with parameters that can be updated in training)





Step function

$$f(s) = \begin{cases} 1, & \text{if } s \geq 0 \\ 0, & \text{if } s < 0 \end{cases}$$

Sign function

$$f(s) = \begin{cases} 1, & \text{if } s \geq 0 \\ -1, & \text{if } s < 0 \end{cases}$$

★ Logistic function

$$f(s) = \frac{1}{1 + e^{-s}}$$

tanh function
小超

$$f(s) = \frac{e^s - e^{-s}}{e^s + e^{-s}}$$

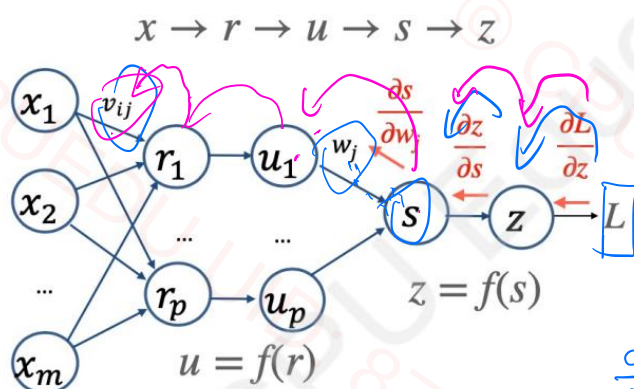
Rectified linear unit

$$f(s) = \max\{0, s\}$$

① tanh 输入饱和
tanh'(x) ≈

② 当 z 很大时
tanh(z) ≈ z

Feedforward Neural Networks: backpropagation

Use chain rule to compute the partial derivatives of loss function with respect to model parameters

$$\frac{\partial L}{\partial w_j} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial s} \cdot \frac{\partial s}{\partial w_j}$$

$$\frac{\partial L}{\partial v_{ij}} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial s} \cdot \frac{\partial s}{\partial u_j} \cdot \frac{\partial u_j}{\partial r_j} \cdot \frac{\partial r_j}{\partial v_{ij}}$$





2.4 Regularization in Deep Neural Networks (DNNs)

• Early Stopping

- Monitoring validation error during training and stopping when the error stops decreasing.
- Prevents overfitting by halting training before the model memorizes noise.
- With some activation functions, early stopping can shrink the DNN towards a linear model.

• Explicit Regularization

Weight Decay

- Analogous to Ridge regression, explicit L2 regularization (weight decay) penalizes large weights:

$$L(\text{data}, \theta) + \lambda \left(\sum_{i=0}^m \sum_{j=1}^p v_{ij}^2 + \sum_{j=0}^p w_j^2 \right)$$

v_{ij} 和 w_j 都搞平

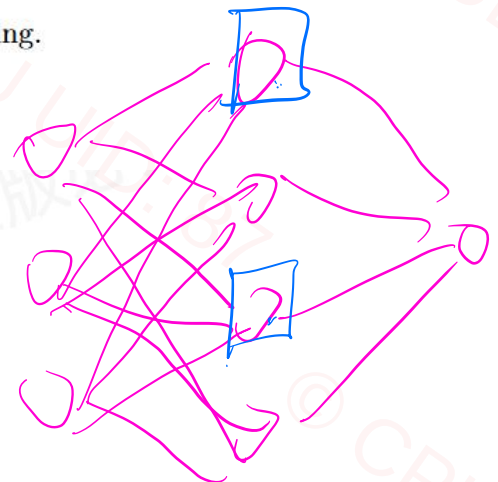
- This adds $2\lambda v_{ij}$ and $2\lambda w_j$ terms to the gradient, controlling the magnitude of weights and preventing overfitting.
- With activation functions like tanh/sigmoid, this also encourages the network to behave more linearly.

• Dropout

- Randomly masks a fraction of units during training, forcing redundancy and ensemble-like behavior.
- Promotes generalization by preventing units from co-adapting.
- Results in smaller weights and less overfitting.
- Used in most state-of-the-art deep learning systems.

训练中

mask掉的unit
不会更新

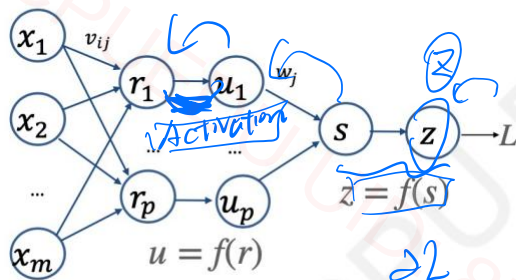


最终使用
是用全部





Given the following neural network for binary classification, if we use binary cross entropy as loss function, compute the partial derivative of loss function L with respect to v_{ij} (written as a formula that only contains x_i , u_j , w_j , z and y , where y is the correct label of data. Assume we use sigmoid activation function everywhere, which are shown as f in the figure below.)



Cross-entropy

$$f(a) = \frac{1}{1+e^{-a}} \quad f'(a) = f(a)(1-f(a))$$

$$L = -[y \ln(z) + (1-y) \ln(1-z)]$$

$$\frac{\partial L}{\partial v_{ij}} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial s} \cdot \frac{\partial s}{\partial u_j} \cdot \frac{\partial u_j}{\partial r_j} \cdot \frac{\partial r_j}{\partial v_{ij}}$$

$$\frac{\partial L}{\partial z} = -\left(\frac{y}{z} - \frac{1-y}{1-z}\right)$$

$$\frac{\partial z}{\partial s} = f'(s) = z(1-z)$$

$$\frac{\partial s}{\partial u_j} = w_j$$

$$\frac{\partial u_j}{\partial r_j} = f'(r_j) = u_j(1-u_j)$$

$$\frac{\partial r_j}{\partial v_{ij}} = x_i$$

$$\frac{\partial L}{\partial v_{ij}} = \left(\frac{y}{z} - \frac{1-y}{1-z}\right) \cdot z(1-z) \cdot w_j \cdot u_j(1-u_j) \cdot x_i$$

$$y = w_0 b_0(x) + w_1 b_1(x) + \dots$$

4. Explain how artificial neural networks can be considered to be a form of non-linear basis function when learning a linear model. [4 marks]

$$h_j(x) = b(w_j^T x + b_j) \quad \text{non-linear basis}$$

ANN 的每个隐藏单元都通过 6 个非线性激活函数进行变换，变换后

$$ANN \text{ 输出层} \quad \hat{y} = \sum_j v_j h_j(x)$$

所以它相当于 Non-linear basis 的 linear model



**Question 13: Neural networks & Regularisation [10 marks]**

- a) Consider the training objective, $\mathcal{L}(\theta) = -\log P_\theta(\mathbf{y}|\mathbf{X}) + \lambda r(\theta)$, where the second term is a *regulariser*, $\lambda > 0$ and $r(\theta) \geq 0$ is a positive-valued function. For the minimiser of the regularised training objective to be a *maximum a posteriori* estimate, state the general form of the corresponding prior over θ . [4 marks]
- b) Given that the output loss of a *feed-forward network*, L , is not a direct function of *hidden-layer weights*, $\mathbf{w}$, how is the derivative of loss with respect to hidden-layer weights, $\frac{\partial L}{\partial \mathbf{w}}$, calculated in *backpropagation*? As part of your answer include the formulation of $\frac{\partial L}{\partial w_j}$ where j is the index of a single weight parameter. [6 marks]

a). Target $\min_{\theta} \mathcal{L}(\theta) = -\log P_\theta(\mathbf{y}|\mathbf{X}) + \lambda r(\theta)$
 $= \max_{\theta} -\mathcal{L}(\theta) = \log P_\theta(\mathbf{y}|\mathbf{X}) - \lambda r(\theta)$
 $\Leftrightarrow \log [P_\theta(\mathbf{y}|\mathbf{X}) \cdot e^{-\lambda r(\theta)}]$

MAP: $\text{Argmax}_{\theta} P(\theta|\mathbf{X}, \mathbf{y}) \propto P(\mathbf{y}|\mathbf{X}; \theta) \cdot P(\theta)$

$$P_\theta(\mathbf{y}|\mathbf{X}) \cdot e^{-\lambda r(\theta)} \propto P(\mathbf{y}|\mathbf{X}; \theta) \cdot P(\theta)$$

$$P(\theta) \propto e^{-\lambda r(\theta)}$$

b). 由于 L 不直接依赖于 w_{ij} , 而是依赖于隐藏层 h_j

$$\frac{\partial L}{\partial w_{ij}} = \frac{\partial L}{\partial h_j} \cdot \frac{\partial h_j}{\partial w_{ij}} = \frac{\partial L}{\partial h_j} \cdot \frac{\partial h_j}{\partial z_j} \cdot \frac{\partial z_j}{\partial w_{ij}}$$

Assume $h_j = b(z_j)$ $z_j = \sum_i w_{ij}x_i + b_j$

$$\frac{\partial L}{\partial w_{ij}} = \frac{\partial L}{\partial h_j} \cdot b'(z_j) \cdot x_i$$





- (a) Consider training a *deep neural network* with *tanh* activation functions using *backpropagation*. Can this lead to the *vanishing gradient problem*? Why? [5 marks]

Yes

① 在DNN中，梯度通过 chain rule 传播

$$\text{eg } \frac{\partial L}{\partial \theta} = \frac{\partial L}{\partial a^{(n)}} \cdot \frac{\partial a^{(n)}}{\partial a^{(n-1)}} \cdots \frac{\partial a^{(1)}}{\partial \theta}$$

但对于 tanh 作为 激活函数， $\tanh'(x)$ 容易出现导数约等于 0
(输入饱和) $\tanh'(x) \approx 0$ ，而一堆的约等于 0 相乘会使
前面整个梯度的约等于 0，这就是 vanishing gradient

- (b) Consider training a *deep neural network* for binary classification. Assume *tanh* activation functions on the hidden layer and training with *backpropagation*. How might this lead to the *vanishing gradient problem*? [3 marks]

- (c) Discuss how a neural network with *tanh* activation functions in its hidden layers can be made into an effectively linear model through regularization. [4 marks]

w_i 很小

$$z = \sum_i w_i x_i$$

输入 z 很小时

$$\tanh(z) \approx z$$

Weight Decay, 惩罚权重过大项

使 NN 权重尽量小

$z = \sum w_i x_i$ 也会变得很小

在 tanh 中，当 z 很小时

$$\tanh(z) \approx z$$

也就意味模型变线性

linear model through regularization





- (d) Consider training a *deep neural network* for predicting temperature given humidity. Which *output* activation would you use and why? Why would *relu* not be appropriate? [3 marks]

temperature 回归问题

线性激活函数

Relu = $\max(0, s)$

输出永远非负，不适合用于预测温度

- (e) While training a neural network model, you observe that the training error initially decreases and but then increases and seems to converge. What has occurred? Provide 2 mechanisms for mitigating this situation and explain why they work. [5 marks]



过拟合

① Early stopping

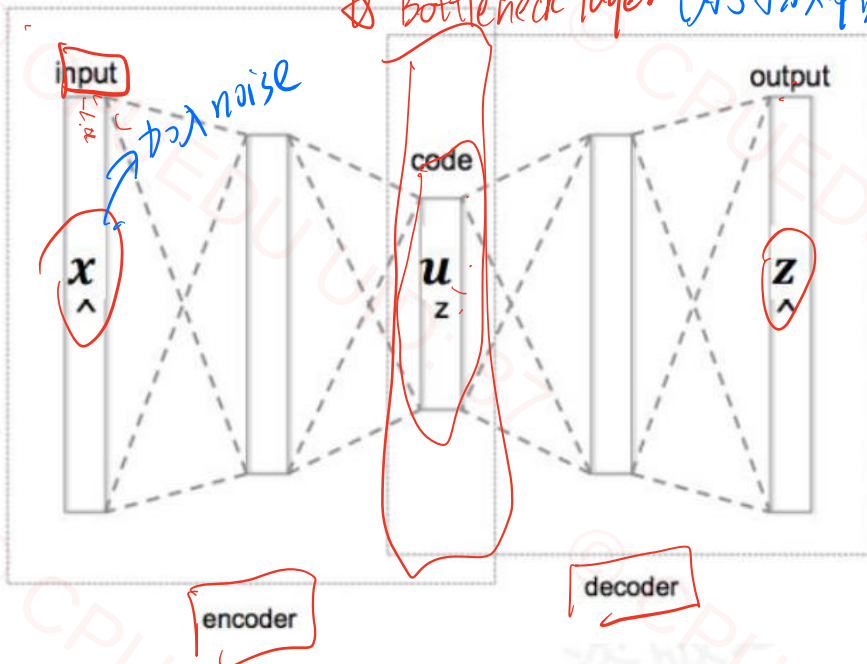
② Weight Decay (正则化)

③ Dropout





PART 2 Autoencoder



Target:

$$z(x_i) \approx x_i$$

Loss:

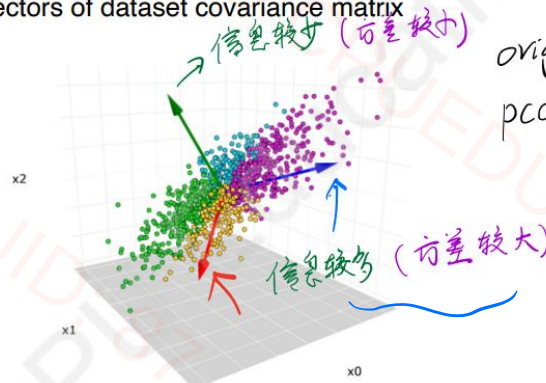
$$L(x, z(x)) = ||x - z(x)||^2$$

undercomplete
overcomplete

Bottleneck层更细；关注压缩，关注学习到最重要特征
更宽，有足够容量，容易直接记住 copy input
没有学习到最重要特征

Autoencoder vs Principle Component Analysis

- Both can be used for dimensionality reduction.
 - PCA finds orthonormal basis where axes are aligned to capture maximum data variation
 - which are eigenvectors of dataset covariance matrix



- Autoencoders (based on neural network-based encoder and decoder) don't explicitly use eigenvalues / eigenvectors





9. With respect to learning an *autoencoder*, explain a failure case that can arise from the use of complex non-linear encoder and decoder components. [4 marks]

过拟合

因为复杂的非线性 Encoder 与 decoder 的学习能力太强，容量很大，网络只是在 copy input，导致了过拟合，没有提取到泛化特征，从而泛化性能差

What is the difference between an undercomplete autoencoder and an overcomplete autoencoder?

undercomplete

bottle neck layer dimension $<$ input data
purpose is to compress input to capture most important feature

overcomplete

dimension $>$ input data

due to large capacity, he is only copying input

What problem does a denoising autoencoder aim to solve, and how does it achieve this?

解决过拟合，提升模型泛化

how? 给 input data 添加 Noise (Corrupt)
然后 encode，尝试重建 uncorrupt data
用



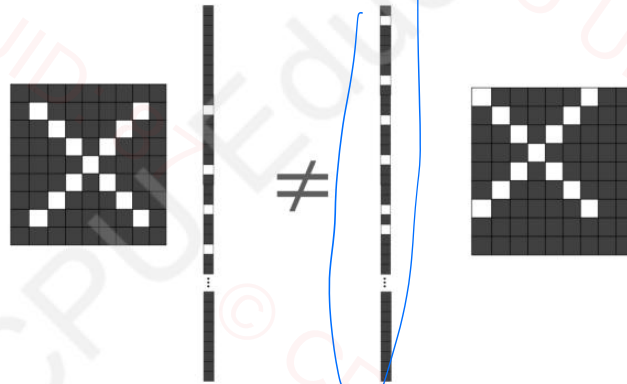


PART 3 CNN

Convolutional Neural Network

Designed for 2-D inputs (image)

- Key advantage compared to Feedforward Neural Networks: Translation invariance



平移不变性

即使图像发生微小平移
也能提取到相似

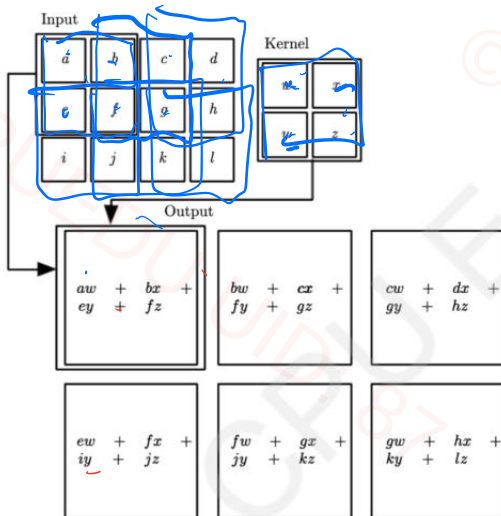
特征

Conv
Pooling

Convolutional Neural Network

Designed for 2-D inputs (image)

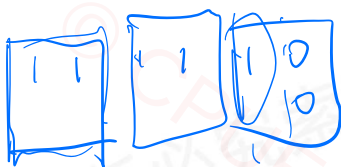
卷积 < ①提取局部特征
②减少参数数量
(减少参数量)



- convolution
- kernel size
- stride
- padding

①保持边缘特征不被模糊
提取
②防止特征图尺寸不断变小

$$H_{out} = \left\lceil \frac{H_{in} + 2 \times h_{padding} - h_{kernel}}{stride} + 1 \right\rceil$$



$$W_{out} = \left\lceil \frac{W_{in} + 2 \times w_{padding} - kernel}{stride} + 1 \right\rceil$$



课程咨询&反馈
请联系课程顾问



Convolutional Neural Network

Designed for 2-D inputs (image)



pooling 出的值可用相同公式计算

pooling

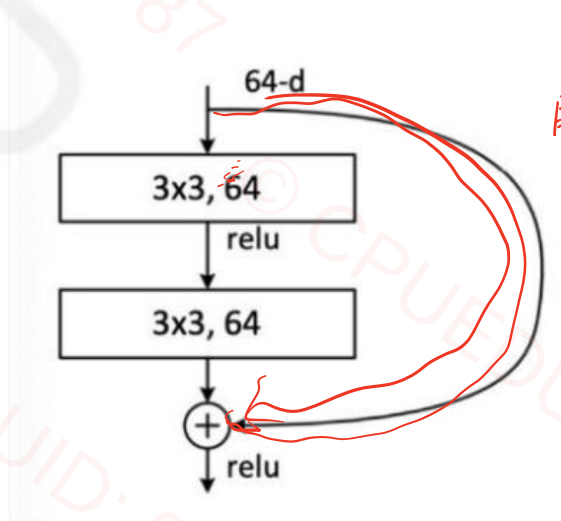
- kernel size 2×2
- stride 2

common pooling methods

- max pooling
- average pooling

$$\left\lceil \frac{4 + 2 \times 0 - 2 + 1}{2} \right\rceil = \lceil 1.5 \rceil = 2$$

Residual Connection

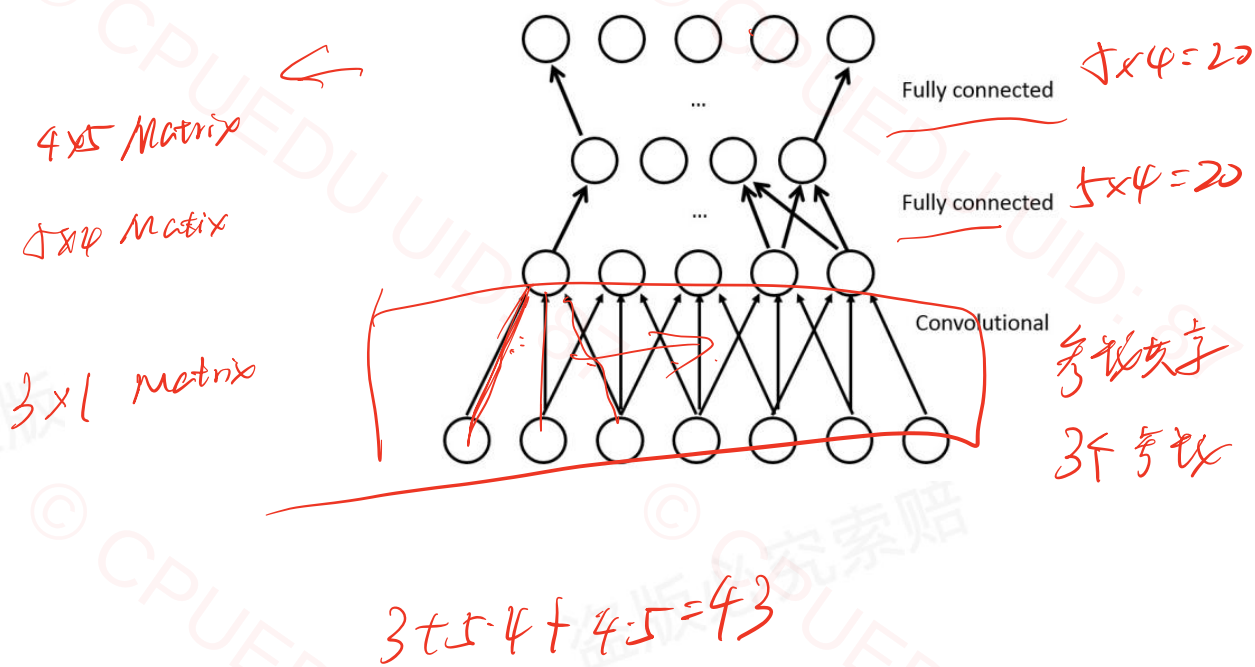


解决 vanish gradient
因为此时 backpropagation
层数多



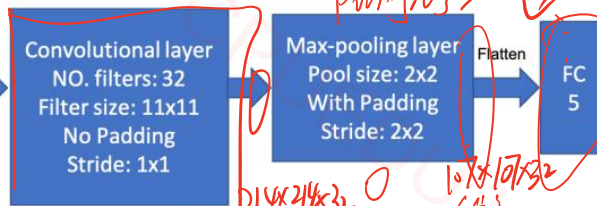


- (d) How many parameters does the following convolutional neural network have (exclude the bias)?
[3 marks]



**Question 6: Convolutional Neural Network [8 marks]**

Consider the following simple *Convolutional Neural Network*. The input is a *RGB image (3 channels)* with width and height equal to 224. The setting of the layers are shown in the figure. There are 32 filters in the convolutional layer and each filter has a size of 11×11 .



- (a) Compute the *number of parameters* in the convolutional layer (show the formula and give the final result) [3 marks]

- (b) In order to reduce the number of parameters of the convolutional layer in the figure, while keeping the receptive field the same, one possible method is to stack one 3×3 convolutional layer and one 9×9 convolutional layer. Describe five different possible stackings, i.e., different combinations of multiple (can be two or more than two) convolutional layers. (Note: changing the order of the multiple convolutional layers is counted as the same stack, e.g., "Stacking a 9×9 and a 3×3 convolutional layer" is counted as the same stack as "Stacking a 3×3 and a 9×9 convolutional layer".) [5 marks]

Each filter $11 \times 11 \times 3 + 1 = 364$

Channel bias

32 filters = $364 \times 32 = 11648$

不包 bias

$11 \times 11 \times 3$ Channel

$32 \times 11 \times 11 \times 3$

总数

$\left\lceil \frac{224 + 2 \times 0 - 11 + 1}{1} \right\rceil = 214$

$\left\lceil \frac{214 - 2 + 1}{2} \right\rceil = 107$

$11648 + 107 \times 107 \times 3 \times 5$



**Question 5: Convolutional Neural Networks [14 marks]**

Consider a simple *Convolutional Neural Network* for processing 3×3 images, with a single filter of size 2×2 , applied with no padding and stride of 1×1 . The next step is *global max-pooling*, which takes the filter output (size 2×2) to produce a single output.

1. What mathematical function is defined by the network, relating its output y to its input $x_{1,1}, x_{1,2}, \dots, x_{3,3}$? You will need to introduce new variables for the model's parameters. [6 marks]
2. Explain how a loss gradient at the output is back-propagated through the network, and show the mathematical form of the loss gradient with respect to the model parameters. [8 marks]

1).

Image:
$$\begin{bmatrix} x_{11} & x_{12} & x_{13} \\ x_{21} & x_{22} & x_{23} \\ x_{31} & x_{32} & x_{33} \end{bmatrix}$$

Assume filter:
$$\begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \end{bmatrix}$$

kernel output

$$\begin{bmatrix} y_{11} & y_{12} \\ y_{21} & y_{22} \end{bmatrix} = \begin{bmatrix} w_{11}x_{11} + w_{12}x_{12} + w_{21}x_{21} + w_{22}x_{22} & w_{11}x_{12} + w_{12}x_{13} + w_{21}x_{22} + w_{22}x_{23} \\ w_{11}x_{21} + w_{12}x_{22} + w_{21}x_{31} + w_{22}x_{32} & w_{11}x_{22} + w_{12}x_{23} + w_{21}x_{32} + w_{22}x_{33} \end{bmatrix}$$

$$y = \max \{ y_{11}, y_{12}, y_{21}, y_{22} \}$$

2).

$$\frac{\partial L}{\partial w_{ij}} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial w_{ij}}$$

Gradient backpropagate only works at maximum index i^*, j^*

$$\frac{\partial L}{\partial y_{i^*, j^*}} \cdot \frac{\partial y_{i^*, j^*}}{\partial w}$$

① if $i^* = 1, j^* = 1$

$$\frac{\partial L}{\partial w_{11}} = \frac{\partial L}{\partial y_{11}} \cdot \frac{\partial y_{11}}{\partial w_{11}} = \frac{\partial L}{\partial y_{11}} \cdot x_{11}$$

$$\frac{\partial L}{\partial w_{12}} = \frac{\partial L}{\partial y_{11}} \cdot \frac{\partial y_{11}}{\partial w_{12}} = \frac{\partial L}{\partial y_{11}} \cdot x_{12}$$

$$\vdots$$

② if $i^* = 1, j^* = 2$

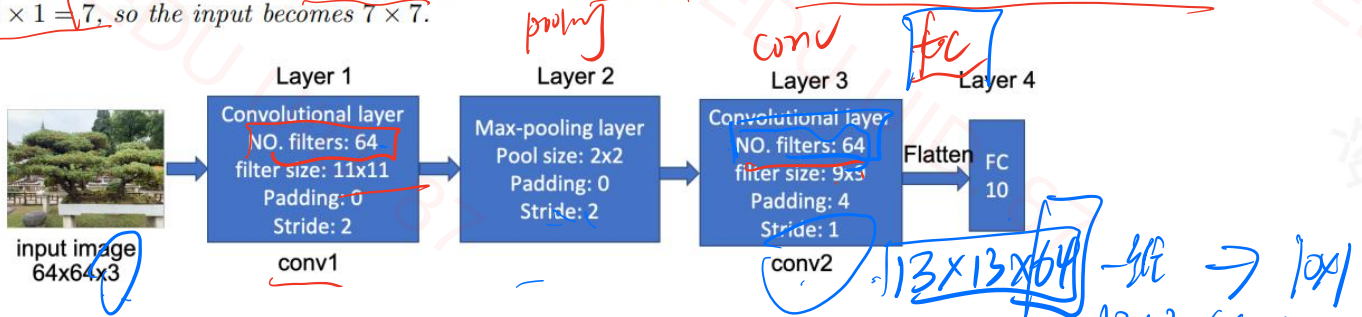
$$\vdots$$



**Question 5: CNN Architectures [16 marks]**

Assume that you train a *convolutional neural network (CNN)* on a GPU. The size of each input image is $64 \times 64 \times 3$ (3 channels). The following figure shows the settings of the layers in the CNN. As shown, there are four layers (blue blocks) in the CNN. Specifically, there are two *convolutional layers* (conv1 and conv2). Between them there is a *max-pooling layer*. The output feature map of conv2 is flattened into a vector, which is then fed to a *fully-connected layer (FC)*. The FC layer contains 10 output units.

Note: The padding size is the number of rows/columns on each size of the input. For example, if the size of an input is 5×5 and the padding size is 1, then after padding, each dimension of the input becomes $5 + 2 \times 1 = 7$, so the input becomes 7×7 .



- (a) Write the sizes of output feature maps of the Layer 1, Layer 2 and Layer 3 (write in the form 'width $\times$ height $\times$ channels') [6 marks]
- (b) How many parameters in each of the four layers (exclude the bias) in the CNN? Show your working. [4 marks]
- (c) How to replace conv2 to keep the same size of receptive field of conv2 with fewer parameters? Write three different replacements. [6 marks]

a).

Layer 1:
$$\left\lceil \frac{64 + 2 \times 0 - 11 + 1}{2} \right\rceil = 27$$

output size: $27 \times 27 \times 64$

Layer 2:
$$\left\lceil \frac{27 + 2 \times 0 - 2 + 1}{2} \right\rceil = 13$$

output size: $13 \times 13 \times 64$

Layer 3:
$$\left\lceil \frac{13 + 2 \times 4 - 9 + 1}{1} \right\rceil = 13$$

output = $13 \times 13 \times 64$ Channel 4

b.

Layer 1 = $11 \times 11 \times 3 \times 64 + 0$

Layer 2 = 0

Layer 3 = $9 \times 9 \times 64 \times 64 + 0$

FC: 输入 $13 \times 13 \times 64$
输出 10
 $13 \times 13 \times 64 \times 10$



**Question 4: Convolutional Operation [10 marks]**

- (a) Assume that in the following figure (a) is an *image patch* and (b) is a *kernel* that is used to perform *convolution* (*stride* = 2, *no padding*) on the patch. Show the *output feature map* after convolution. [4 marks]

1	0	1	0
0	1	1	0
0	0	0	1
0	0	1	0

(a)

1	0
0	1

(b)

$$\begin{bmatrix} 2 & 1 \\ 0 & 0 \end{bmatrix}$$

- (b) Assume that (a) and (b) in the following figure are two image patches. Now consider a 3×3 *binary weight matrix* (each element of the matrix is either 0 or 1) that is used as the kernel to perform *convolution* (*stride* = 1, *no padding*) on the two image patches. Assume that after convolution, the output feature maps are K_a and K_b . What kernel can make the difference between K_a and K_b most significant (sum of all elements of $|K_a - K_b|$ is maximum)? List two such kernels. [6 marks]



0	1	0
1	1	1
0	1	0

(a)

1	0	1
0	1	0
1	0	1

(b)

$$\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

①

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & 1 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

$$a = 5$$

$$b = 4$$

$$2 \times 1$$

$$\text{for } a = 5$$

$$\text{for } b = 1$$

$$= 4$$

$$\begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix} \quad \begin{matrix} \text{for } a = 1 \\ \text{for } b = 5 \end{matrix}$$



**Question 5: Convolutional Neural Networks [14 marks]**

Consider the following toy *convolutional neural network (CNN)*: a single-channel 32×32 pixel input image (first blue square) is first fed into a *convolutional layer* (first black square) resulting in a 15×15 output (second blue square), which is then fed into a *max-pooling layer* (second black square) resulting in an output (third blue square) of unknown dimension.



Prior to the *convolutional layer*, each of the four sides of the image is *padded* by 1 pixel. This layer has a single 5×5 filter/kernel with some unknown *stride*. The *max-pooling* layer uses a 2×2 pool size, with 2×2 stride and no padding.

- (a) What is the *convolutional layer's* missing *stride* value? [8 marks]
 (b) What is the missing final *feature map dimension*? [6 marks]

$$a). \quad \left\lceil \frac{32 + 2 \times 1 - 5}{\text{stride}} + 1 \right\rceil = 15$$

$$\left\lceil \frac{30}{\text{stride}} \right\rceil = 15 \quad \text{stride} = 2$$

$$b). \quad \left\lceil \frac{15 - 2 + 1}{2} \right\rceil = 7$$

$$7 \times 7$$



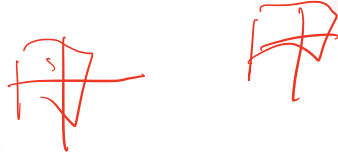


(c) Explain how *convolutional neural networks (CNNs)* can produce *translation invariant* representations when applied to vector or matrix inputs. [5 marks]

① Convolution operation, 卷积

允许 CNN 在不同的 location 检测到相似的特征

(如果因为图像平移)



② Pooling operation

aggregate features within small local region

so small shifts in input cause no changes in output

(b) What are two benefits of *max pooling*. [5 marks]

① can see translation invariant

features from input

② reduce dimension of feature map

(c) *Padding* can be used to make the output and input dimensions of a *convolutional layer* the same. What is another use of *padding*? [5 marks]

better representation for pixels in corner of the image

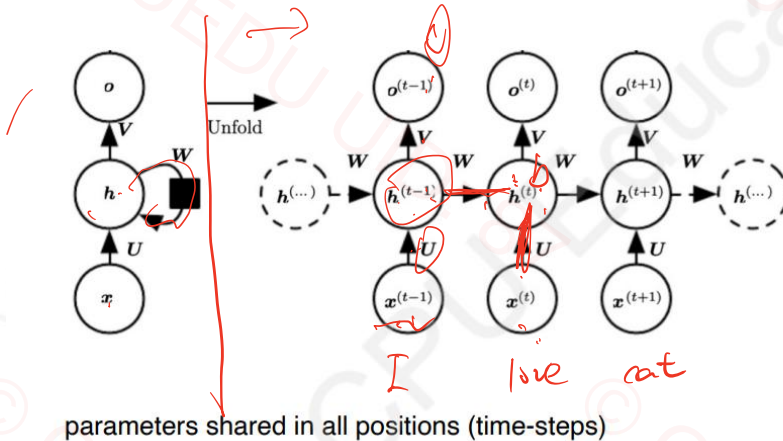




PART 4 RNN

Sequence Model: Recurrent Neural Network

For processing sentence (a sequence of words), audio clip (sequence of time steps) and video clip (sequence of frames)



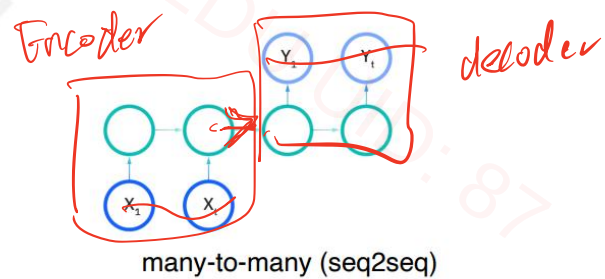
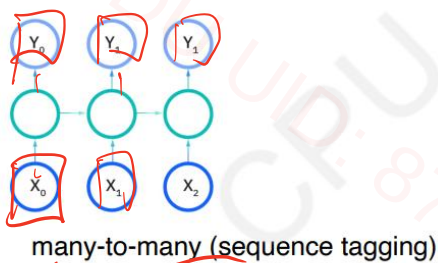
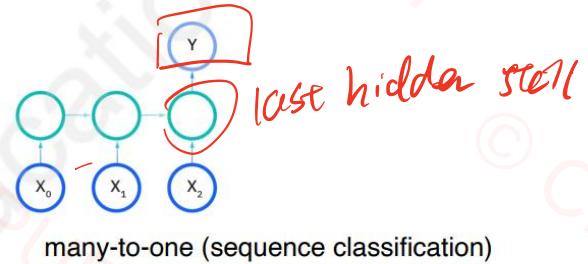
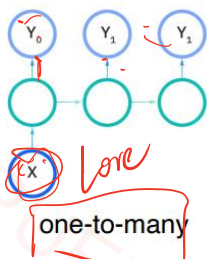
$$\begin{aligned} a^{(t)} &= b + Wh^{(t-1)} + Ux^{(t)}, \\ \tilde{h}^{(t)} &= \tanh(a^{(t)}), \\ o^{(t)} &= c + V\tilde{h}^{(t)}, \end{aligned}$$

每个 hidden state 都依赖于
当前输入及上一隐藏状态

W, U, b, V, c

参数共享性

Sequence Model: Recurrent Neural Network Architectures



翻译, 对译

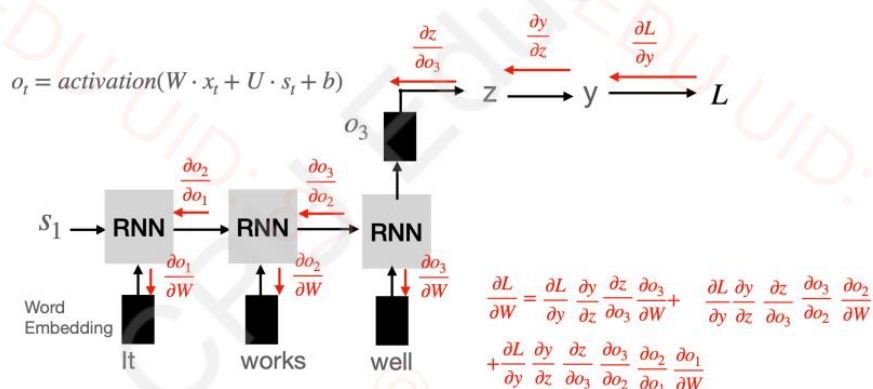




Sequence Model: Recurrent Neural Network

Use backpropagation through time (BPTT) to update parameters (propagate gradients back to the very start of the network).

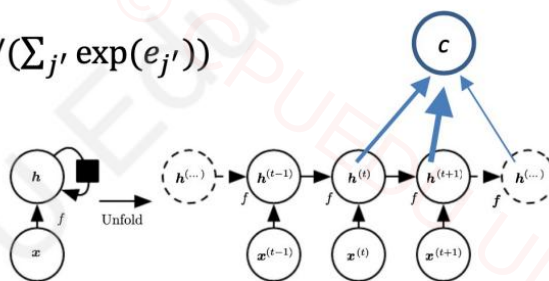
Gradient vanishing/explosion problem: if too many time-steps, gradients can be very small / large as a result of multiple multiplications. Can't learn long distance phenomena (10+ steps).



Sequence Model: Attention

Take a weighted average of hidden sequence instead of focusing on later steps

- * $c = \sum_j \alpha_j h^{(j)}$
- * $\alpha_j = \exp(e_j) / (\sum_{j'} \exp(e_{j'}))$
- * $e_j = f(h^{(j)})$





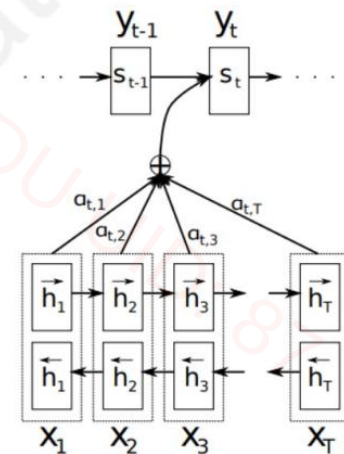
Sequence Model: Attention in Seq2Seq models

Avoids information bottleneck problem

$$\mathbf{c}_i = \sum_j \alpha_{ij} \mathbf{h}^{(j)}$$

$$\alpha_{ij} = \exp(e_{ij}) / (\sum_{j'} \exp(e_{ij'}))$$

$$e_{ij} = f(\mathbf{s}^{(i)}, \mathbf{h}^{(j)})$$



1. A *recurrent neural network* is as a type of *deep artificial neural network*. In what respect is it *deep*? [4 marks]

- (d) For the *recurrent neural network (RNN)* that takes a sequence as the input, explain why we need to use *backpropagation through time* to update weights. [5 marks]





(b) Both *convolutional networks (CNNs)* and *recurrent networks (RNNs)* can be applied to sequence inputs. Explain the key benefit that RNNs have over CNNs for this type of input. [5 marks]

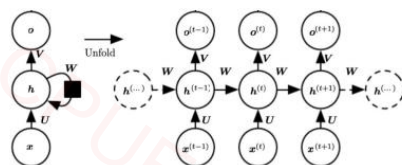
(c) Explain in words how *Attention* can be used to allow for neural models to process dynamic sized inputs. [5 marks]

(b) Why are *vanishing gradients* a more pressing concern for *recurrent neural networks (RNNs)* than for other neural architectures? [5 marks]





How many parameters are there in total? Suppose the dimension of x , h and o are d_1 , d_2 and d_3 , respectively.



$$\begin{aligned} \mathbf{a}^{(t)} &= \mathbf{b} + \mathbf{W}\mathbf{h}^{(t-1)} + \mathbf{U}\mathbf{x}^{(t)}, \\ \mathbf{h}^{(t)} &= \tanh(\mathbf{a}^{(t)}), \\ \mathbf{o}^{(t)} &= \mathbf{c} + \mathbf{V}\mathbf{h}^{(t)}, \end{aligned}$$

Why is the Transformer model able to process sequences in parallel, unlike RNNs?





